CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering
Devang Patel Institute of Advance Technology and Research (DEPSTAR)
Data Engineering (ITUC301)
Practical File – ODD Semester 2026–27
Student ID: 24DIT057
Student Name: PRINCE V. RAIYANI

PRACTICAL 2

Practical Definition: Programmatically interface with diverse file-based, API-driven, and relational database systems. Learners will build data generation engines to simulate transactional workloads, profile incoming files for automated schema discovery, isolate data quality violations (such as null leaks and format divergence), and establish robust operational ingestion criteria.

Solution :

Full project structure…

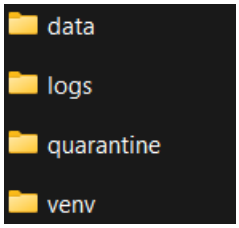
```
data_ingestion_practical/
│
├── data/
│   ├── customers.csv
│   ├── api_data.json
│   ├── config.txt
│
├── quarantine/
│   └── (invalid records)
│
├── logs/
│   └── validation.log
│
├── generate_data.py
├── api_fetch.py
├── profile_data.py
├── validate_data.py
├── database.py
├── requirements.txt
└── README.md
```

Step 1:

Check Python Version .

```
D:\>python --version
Python 3.14.2
```

Step 2: folders



Step 3: create venv

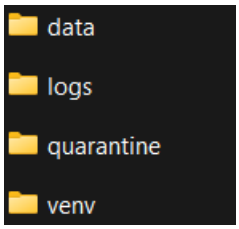
Step 4: create Require lib

```
D:\SEM5\Data Engineering\Data_Ingestion_Practical>python -m venv venv

D:\SEM5\Data Engineering\Data_Ingestion_Practical>venv\Scripts\activate

(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>pip install faker pandas requests
```

Step 5 : Folder Structure



Step 6 : Generate Customer Data

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python generate_data.py
   customer_id            name                      email  age
0            1      Cory Davis     tmitchell@example.net   38
1            2   Shelia Conner   kellydeanna@example.org   55
2            3  Javier Carroll   millsdebbie@example.com   25
3            4      Lisa Hayes  brittneyhill@example.org   50
4            5    Carrie Leach        dsmith@example.org   29
Customer CSV Generated Successfully
```

Step 7 : Understand CSV file

customer_	name	email	age
1	Cory Davis	tmitchell@	38
2	Shelia Con	kellydeann	55
3	Javier Carr	millsdebbi	25
4	Lisa Hayes	brittneyhill	50
5	Carrie Lea	abc.com	29
6	Christophe	weaverana	46
7	Savannah	qlopez@e	31

Step 8 : Create Bad Data

A	B	C	D
customer_	name	email	age
21		preet@example.com	654

Step 9 : Profile dataset

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python profile_data.py

Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   customer_id  20 non-null     int64
 1   name         20 non-null     str
 2   email        20 non-null     str
 3   age          20 non-null     int64
dtypes: int64(2), str(2)
memory usage: 772.0 bytes
None

First Five Rows
   customer_id            name                      email  age
0            1      Cory Davis     tmitchell@example.net   38
1            2   Shelia Conner   kellydeanna@example.org   55
2            3  Javier Carroll   millsdebbie@example.com   25
3            4      Lisa Hayes  brittneyhill@example.org   50
4            5    Carrie Leach        dsmith@example.org   29

Missing Values
customer_id    0
name           0
email          0
age            0
dtype: int64

Data Types
customer_id    int64
name             str
email            str
age            int64
dtype: object

Duplicate Rows
0
```

Step 10 : Validate Data

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python validate_data.py
Validation Completed
Valid Records: 20
Invalid Records: 0
```

Step 11 : Logging Errors

```
2026-07-30 12:55:20,591 - INFO - Validation completed successfully
2026-07-30 12:56:05,691 - ERROR - Row 21: Name is missing
2026-07-30 12:56:05,696 - INFO - Validation completed successfully
```

Step 12 : Read API

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python api_fetch.py
[{'id': 1, 'name': 'Leanne Graham', 'username': 'Bret', 'email': 'Sincere@april.biz', 'ad
nborough', 'zipcode': '92998-3874', 'geo': {'lat': '-37.3159', 'lng': '81.1496'}}, 'phone
{'name': 'Romaguera-Crona', 'catchPhrase': 'Multi-layered client-server neural-net', 'bs'
```

Step 13 : Read JSON

```
import pandas as pd
df=pd.read_json("data/api_data.json"
print(df.head())
```

Step 14 : Configuration File

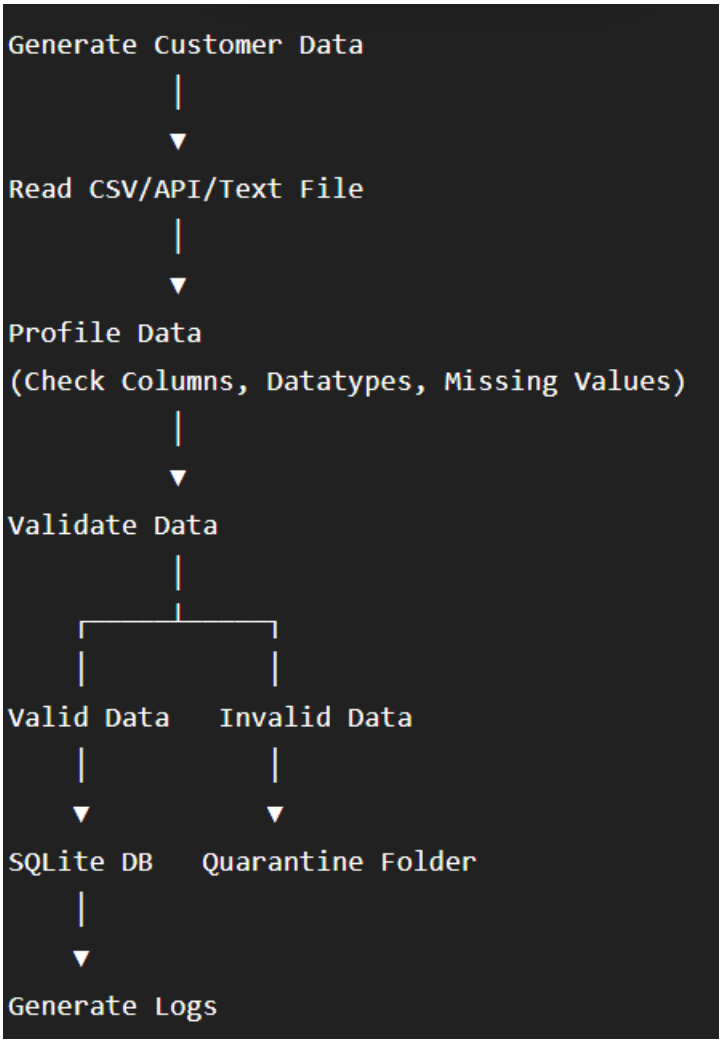
ata > config.txt

```
1  Server=localhost
2  Port=5432
3  Database=customer_db
4  Username=admin
5  Password=12345
6  Debug=True
```

Step 15 : SQLite Database

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python database.py
Database Created
```

Step 16 : Final Workflow



Read JSON :

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python read_json.py
First 5 Records:
   id             name  ...        website                                                company
0   1     Leanne Graham  ...  hildegard.org  {'name': 'Romaguera-Crona', 'catchPhrase': 'Mu...
1   2      Ervin Howell  ...  anastasia.net  {'name': 'Deckow-Crist', 'catchPhrase': 'Proac...
2   3  Clementine Bauch  ...     ramiro.info  {'name': 'Romaguera-Jacobson', 'catchPhrase': ...
3   4  Patricia Lebsack  ...        kale.biz  {'name': 'Robel-Corkery', 'catchPhrase': 'Mult...
4   5  Chelsey Dietrich  ...    demarco.info  {'name': 'Keebler LLC', 'catchPhrase': 'User-c...

[5 rows x 8 columns]

Columns:
Index(['id', 'name', 'username', 'email', 'address', 'phone', 'website',
       'company'],
      dtype='str')

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
```

Extract JSON :

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python extract_json.py
Customer Name:
Leanne Graham
```

Read Config.py :

```
(venv) D:\SEM5\Data Engineering\Data_Ingestion_Practical>python read_config.py
Configuration File Contents:

Server=localhost
Port=5432
Database=customer_db
Username=admin
Password=12345
Debug=True
```

Key Question :

Q1. Explain the architectural strategies used to manage severe structural schema drift when handling continuous external third-party API inputs.

Answer:
To manage schema drift, the system first identifies the structure of incoming data and compares it with the expected schema. If new fields or changes are detected, they are handled without interrupting the ingestion process. Invalid or unexpected records are stored separately, allowing the system to continue processing valid data while maintaining flexibility.

Q2. What are the structural benefits and data storage trade-offs of using semi-structured nested JSON objects over heavily normalized SQL database records?

Answer:
Nested JSON makes it easier to store complex and changing data because it does not require a fixed table structure. It is commonly used for API responses and supports hierarchical information naturally. However, it may use more storage space and can be less efficient for searching and joining data than a normalized SQL database.

Q3. How do you validate data quality at the point of ingestion without creating severe processing performance bottlenecks?

Answer:
Data quality can be checked by applying simple validation rules such as checking for missing values, correct data types, and valid formats during data ingestion. Records that fail these checks are moved to a separate location for review instead of stopping the entire process. This approach keeps the system efficient while ensuring that only reliable data is processed further.

Conclusion :

This practical provided hands-on experience in collecting data from different sources and preparing it for further use. It demonstrated how to inspect, validate, and organize incoming data before storing it in a database. Overall, the implementation helped develop practical knowledge of data ingestion, quality checking, and basic data engineering techniques.